

Multi-Agent Systems in Modern Software Engineering

Muhammad Swalha and Mohammed Abad

Course: Mass Production of AI Agents

Lecturer: Dr. Yoram Segal

Contents

1	Introduction — The Rise of Collaborative AI in Software Development	2
2	Definitions, Core Concepts, and the LLM-Based Multi-Agent Paradigm	3
3	Architectural Patterns — From Single Orchestrators to Distributed Teams	4
3.1	Framework Comparison	5
4	Capabilities and Use Cases Across the Software Development Life Cycle	5
5	Failure Modes — Why Multi-Agent Systems Break Down	7
6	Engineering Best Practices for Reliable Multi-Agent Systems	9
7	Domain Applications — From Financial Services to Production Incident Response	11
8	Trends, Evolutionary Stages, and the Changing Role of the Software Engineer	12
9	Synthesis — Building MAS-Ready Engineering Organizations	14
10	Hebrew Summary / סיכום בעברית	16
11	Data Visualizations	16
12	Block Diagram	16
	References	18

1 Introduction — The Rise of Collaborative AI in Software Development

Software engineering has always been fundamentally a collaborative discipline. Teams of developers, architects, testers, and project managers work in concert to transform abstract requirements into functioning systems. For decades, the aspiration of automating parts of this process drove research into code generation, static analysis, and automated testing. Yet until recently, these efforts remained siloed — a tool that generated code could not review it, and a tool that reviewed code could not deploy it. The emergence of Large Language Models (LLMs) capable of reasoning, generating, and critiquing code changed the landscape dramatically, but even a single powerful LLM operates within the limitations of a fixed context window and a single chain of thought. What software engineering truly needed was not a smarter solitary tool, but something closer to a smarter team.

Multi-agent systems (MAS) represent precisely that conceptual leap. At their core, a MAS works by distributing tasks and communication among individual agents, each working together to achieve a goal within a shared environment [12]. Rather than relying on one monolithic model to handle every aspect of a software project — from parsing requirements through deploying a container — a MAS assigns discrete responsibilities to specialized agents and coordinates their outputs into a coherent product. The analogy to human engineering organizations is not accidental; it is deliberate architectural thinking that draws on decades of organizational theory and computer science research alike.

What makes this moment historically significant is the confluence of two trends. First, LLMs have reached a threshold of language and reasoning capability that makes them credible participants in software engineering workflows. Second, orchestration infrastructure — frameworks, APIs, and tooling — has matured to the point where connecting multiple LLM-powered agents into a reliable pipeline is an engineering challenge that teams can actually solve. In modern software engineering, MAS agents are described as “specialized AI workers with defined roles, tools, and accountabilities — context-aware, purpose-driven, and capable of coordinating with other agents” [9]. This framing positions agents not as replacements for engineers but as members of an AI-augmented engineering organization.

This article surveys the state of multi-agent systems in modern software engineering across seven substantive dimensions: foundational definitions and concepts, architectural patterns and frameworks, practical capabilities and use cases, documented failure modes and systemic challenges, engineering best practices, domain-specific applications, and future trajectories. Throughout, the goal is to give software engineers, architects, and technical leaders an accurate, nuanced, and actionable understanding of MAS — not as a speculative future technology but as a present engineering discipline with real trade-offs and real promise.

The structure of what follows reflects the complexity of the subject. MAS in software engineering is not a single technique; it is an architectural paradigm with multiple instantiations, each suited to different problem shapes. Understanding when to deploy a multi-agent architecture, how to design one that is reliable, and how to avoid the failure modes that have already been documented in production and research settings is the practical payoff of this investigation. We begin at the conceptual foundations.

2 Definitions, Core Concepts, and the LLM-Based Multi-Agent Paradigm

Before examining architectures and applications, it is important to establish a precise vocabulary. The term "agent" in software engineering long predates large language models. In classical AI, an agent is any entity that perceives its environment and takes actions to maximize some objective. In MAS research, these agents interact — cooperating, competing, or negotiating — to produce system-level behaviors that no single agent could achieve alone. What the LLM era has contributed is a new class of agents with dramatically enhanced natural-language understanding, generative capability, and commonsense reasoning — properties that make them useful participants in software engineering workflows that require reading and writing code, understanding specifications, and communicating findings.

LLM-Based Multi-Agent (LMA) systems integrate Large Language Models into multi-agent architectures to address complex, multi-faceted software engineering challenges such as requirements engineering, code generation, software quality assurance, and maintenance [8]. The integration is consequential: LLMs give agents the ability to process unstructured natural language inputs (like a requirements document or a bug report), generate structured outputs (like code or a test plan), and engage in dialogue with other agents using the same medium — natural language — that human teams use to coordinate. This creates a shared semantic layer that earlier rule-based or symbolic MAS architectures lacked.

Individual agents within a MAS are typically equipped with four core components. First, a **role or persona** that scopes the agent's responsibilities and shapes its reasoning. A Coder Agent is prompted to think as a software engineer; a Reviewer Agent is prompted to think as a code critic. Second, **memory** — both short-term (within-context) and long-term (external vector stores or databases) — that allows agents to retain information across interactions. Third, a **toolset**: concrete capabilities such as a code interpreter, a web search API, a file system interface, or a terminal emulator that agents use to interact with the world beyond their context window. Fourth, **communication protocols** that govern how agents send messages to one another, how they signal task completion, and how they escalate failures [8].

These components combine to produce agents that are substantially more capable than prompt-engineering a single LLM. A single LLM asked to both write and review code is subject to self-confirmation bias — it tends to validate its own outputs. An agent architecture that separates the Coder role from the Reviewer role, and instantiates each as a distinct LLM call with a distinct persona and instruction set, structurally counteracts that bias. Similarly, a single LLM cannot hold an entire large codebase in its context window, but a MAS can distribute ownership across agents, each maintaining deep context over its assigned module while communicating summaries to coordinating agents.

The conceptual distinction between a multi-agent system and a multi-step pipeline is worth drawing carefully. A pipeline is a sequence of deterministic operations; its behavior is fully specified at design time. A MAS involves agents that perceive, reason, and decide — introducing a degree of autonomy that enables more flexible behavior but also introduces new failure modes. This distinction has practical engineering consequences: pipelines are easier to test and debug, while MAS are more capable on open-ended tasks but require more sophisticated observability infrastructure. Understanding this trade-off is

the starting point for responsible MAS engineering.

3 Architectural Patterns — From Single Orchestrators to Distributed Teams

The architecture of a multi-agent system in software engineering is not one-size-fits-all. Different problem structures demand different coordination patterns, and the choice of architecture has significant implications for reliability, scalability, and debuggability. Three broad architectural patterns have emerged as dominant in contemporary practice, and several purpose-built frameworks have been designed to implement each.

The most common pattern in production MAS is the **orchestrator-worker hierarchy**. A central Planner or Orchestrator Agent receives the top-level task — for example, “implement the authentication module for this web application” — and decomposes it into subtasks that are delegated to specialist Worker Agents. A typical pipeline includes a Planner Agent, a Coder Agent, a Reviewer Agent, and a Tester Agent, each handling discrete phases of development [6]. The Orchestrator maintains the master task state, tracks which subtasks have been completed, and manages dependencies between them. This pattern offers strong auditability because all coordination flows through a single node, but it also creates a bottleneck: if the Orchestrator fails or makes a poor decomposition decision, downstream agents inherit the mistake.

A second pattern is **peer-to-peer collaboration**, in which agents communicate directly without a central coordinator. This is more common in research settings and in systems that need to simulate deliberation — for example, a requirements engineering MAS in which a Business Analyst Agent, a Technical Architect Agent, and a Security Compliance Agent negotiate the feasibility of proposed requirements. The AutoGen framework from Microsoft excels in this conversational multi-agent paradigm, enabling agents to engage in extended back-and-forth dialogue and reach consensus through structured conversation [7]. The cost of this flexibility is coordination complexity: without a central arbiter, agents can deadlock, produce conflicting outputs, or fail to converge on a decision.

A third pattern is the **event-driven MAS**, in which agents subscribe to event streams and react to triggers rather than receiving explicit task assignments. This architecture is particularly relevant for operational software engineering tasks — monitoring production systems, detecting anomalies, and triggering incident response workflows. Gartner has highlighted that “the success of multiagent generative systems will require specialized software engineering with a data-centric, event-driven architecture” — underscoring that real-time context is critical for agents that must act on live system state [14]. Event-driven MAS are more decoupled and horizontally scalable than orchestrated hierarchies, but they require robust event schema design and idempotent agent handlers to avoid cascading errors.

The three dominant open-source orchestration frameworks for MAS in software engineering — LangGraph, AutoGen, and CrewAI — each embody different points in this design space. **LangGraph** is preferred for production-grade control and complex state management, providing a graph-based abstraction in which nodes represent agent steps and edges represent conditional transitions. **AutoGen** (Microsoft) excels in conversational multi-agent systems and Azure-integrated environments. **CrewAI** simplifies collaborative task execution with structured roles and agent hierarchies, lowering the barrier to multi-agent prototyping [7]. The choice between them is not purely technical; it is also

organizational. A team with deep Azure investment will find AutoGen’s integrations valuable. A team prioritizing rapid prototyping of role-based workflows may prefer CrewAI’s declarative API. A team building a production system that requires fine-grained control over execution flow will likely gravitate toward LangGraph.

3.1 Framework Comparison

The following table summarizes the key characteristics of the three dominant MAS frameworks to assist practitioners in selecting the most appropriate tool for their context.

Framework	Primary Paradigm	Best For	State Management	Production Readiness	Ecosystem
LangGraph	Graph-based flow	Complex, stateful production pipelines	Fine-grained, explicit	High	Python-native; broad LLM support
AutoGen (Microsoft)	Conversational agents	Dialogue-heavy workflows; Azure environments	Moderate; session-scoped	High	Deep Azure / Microsoft integration
CrewAI	Role-based crew	Rapid prototyping; role-defined tasks	Declarative; abstracted	Moderate	Growing; Python-native

Two formal frameworks have been proposed specifically for software engineering MAS. The **ALMAS framework** (Autonomous LLM-based Multi-Agent Software Engineering) proposes covering the full Software Development Life Cycle (SDLC): requirements analysis, design, implementation, testing, and deployment [1]. ALMAS is notable for its ambition — most MAS implementations address one or two phases of the SDLC, but ALMAS posits an end-to-end autonomous engineering system. The **SALLMA architecture** (published April 2025) provides a formal software architecture blueprint for LLM-based multi-agent systems, addressing modularity, scalability, and inter-agent communication as first-class design concerns. Together, these frameworks signal that the research community is moving from ad hoc MAS implementations toward principled software architectures for agent systems — a maturation that parallels the historical development of software architecture patterns for distributed systems.

4 Capabilities and Use Cases Across the Software Development Life Cycle

The value proposition of multi-agent systems in software engineering rests on two foundational capabilities: specialization and parallelism. Specialization enables each agent to develop a deep, focused competency within its domain — a Reviewer Agent that has been prompted with comprehensive code review heuristics will produce more thorough feedback than a general-purpose LLM asked to review code as an afterthought. Parallelism enables

multiple agents to work concurrently on independent tasks, overcoming the sequential bottleneck inherent in single-agent workflows. MAS can parallelize software engineering tasks that would bottleneck a single model — including simultaneous code generation across multiple modules, concurrent test execution, and parallel documentation generation [8].

The throughput advantage of parallelism can be expressed formally. If a pipeline of n independent tasks each requires time t_i , the theoretical minimum wall-clock time under full parallelism is:

$$T_{\text{parallel}} = \max_{i=1}^n t_i \quad (1)$$

compared with the sequential baseline:

$$T_{\text{sequential}} = \sum_{i=1}^n t_i \quad (2)$$

In practice, coordination overhead and data dependencies prevent perfect parallelism, but the fundamental speedup potential — proportional to the ratio of the sequential sum to the maximum individual task duration — is the core quantitative argument for MAS in large-scale software development workflows.

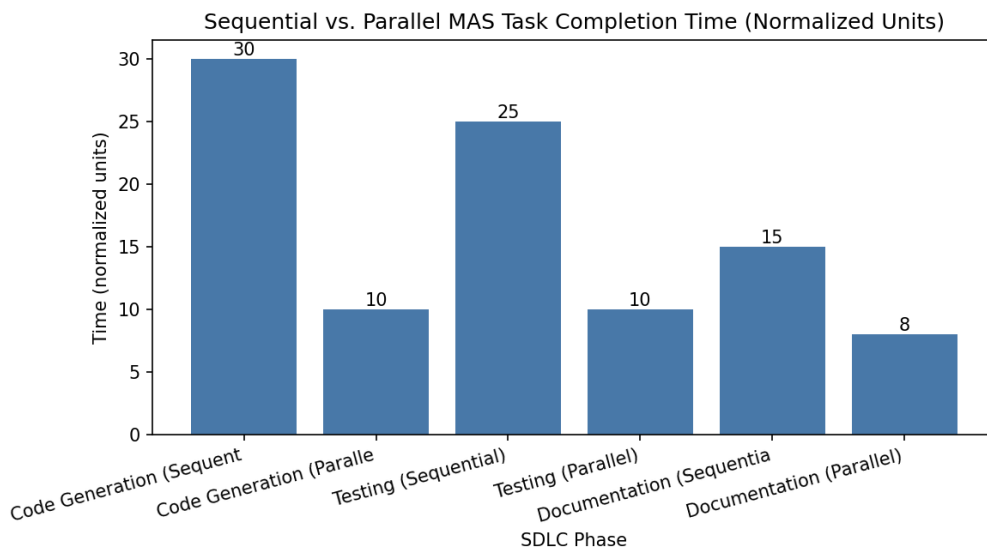


Figure 1: Bar chart comparing sequential vs. parallel MAS task completion times across code generation, testing, and documentation phases of the SDLC

In **requirements engineering**, MAS offers qualitatively new capabilities. Traditional requirements management tools are largely passive — they store and track requirements but do not reason about them. A MAS-based requirements engineering system can deploy agents that clarify ambiguous requirements through dialogue with human stakeholders, decompose high-level requirements into implementable user stories, check for logical contradictions, and validate requirements against regulatory frameworks. The conversational capability of LLM-based agents is particularly valuable here: requirements are fundamentally expressed in natural language, and an agent that can ask clarifying questions, propose alternative interpretations, and synthesize consensus from multiple stakeholder inputs brings a qualitatively new kind of automation to this SDLC phase.

Code generation and review is the most mature application domain for MAS in software engineering. Dedicated Coder Agents can generate code from structured task descriptions, while Reviewer Agents iterate on code quality — checking for correctness, security vulnerabilities, performance issues, and adherence to project coding standards [8]. The iterative loop between Coder and Reviewer Agents simulates the pull-request review cycle that human engineering teams rely on, but can run at machine speed and scale. In practice, this means a MAS can generate and review dozens of code modules in parallel, with each module receiving dedicated reviewer attention rather than the cursory review that time-pressured human engineers sometimes provide.

Software quality assurance is another domain where MAS capabilities are particularly compelling. Autonomous test generation — having an agent read code, infer its intended behavior, and write test cases that exercise that behavior — is a long-standing aspiration of software engineering research. MAS embeds this capability in a continuous workflow: a Test Writer Agent generates test cases, a Test Executor Agent runs them against the current codebase, and a Bug Reporter Agent synthesizes failures into actionable reports for the Coder Agent. Agents that autonomously write and execute test suites can achieve coverage levels that would be prohibitively expensive to produce manually [8]. The memory components of these agents are critical: a Bug Reporter Agent with access to historical bug reports can identify patterns — for example, that authentication-related bugs cluster around a particular module — and direct test generation efforts accordingly.

Maintenance and debugging rounds out the key use cases. Production software systems generate continuous streams of logs, metrics, and alerts, and the engineering challenge of making sense of these streams in real time is substantial. MAS architectures with memory streams that track system state across time are well-suited to this challenge. Resolve AI has built production multi-agent systems for engineers that “maintain investigation context” on live production systems, enabling autonomous incident response — where agents diagnose the root cause of production incidents, propose remediation actions, and in some cases execute those actions autonomously [11]. This represents a qualitative shift in the role of on-call engineers: rather than manually correlating logs across systems, an engineer can review the diagnostic narrative produced by an autonomous investigation agent and focus their judgment on the remediation decision.

The domain of financial services illustrates how the specialization and accountability properties of MAS translate to regulated industries. Multi-agent software engineering enables specialized AI workers to manage complex, compliance-sensitive workflows with accountability and auditability [9]. In this context, the agent architecture provides not just technical benefits but organizational benefits: each agent’s decisions and tool calls are logged, creating an audit trail that satisfies regulatory requirements. A Compliance Review Agent can check generated code against regulatory policy documents before deployment, ensuring that no code change bypasses compliance review — a function that is difficult to enforce reliably in purely human-managed workflows.

5 Failure Modes — Why Multi-Agent Systems Break Down

The capabilities described in the previous chapter are real, but they exist in productive tension with a set of documented failure modes that are equally real. A balanced understanding of MAS in software engineering requires honest engagement with these failures

— not to dismiss the technology but to understand the engineering challenges that must be solved before MAS can be reliably deployed in high-stakes production environments.

A 2025 study analyzing five popular MAS frameworks across 150+ tasks found that despite widespread enthusiasm, performance gains on popular benchmarks are often minimal [13]. This finding challenges a naive assumption: that simply adding more agents must improve performance. In practice, adding agents adds coordination overhead, introduces new failure points, and can produce compounding errors when one agent’s mistake propagates as input to the next. The benchmark analysis revealed that much of the perceived benefit of MAS in informal demonstrations does not survive rigorous systematic evaluation.

The same study identified 14 unique failure modes organized into three categories [13]. The first category, **Specification Issues**, covers failures rooted in how agents and their tasks are defined: poorly specified agent roles, ambiguous task descriptions, and unclear success criteria. These are design-time failures that often manifest at runtime in subtle ways — an agent with an underspecified role may behave competently on easy tasks and fail silently on edge cases. The second category, **Organizational and Coordination Issues**, covers failures in how agents interact: task handoff failures, conflict resolution breakdowns, shared state corruption, and deadlocks in peer-to-peer systems. The third category, **Agent Execution Issues**, covers individual agent failures: hallucination, tool misuse, context loss, and failure to follow structured output formats.

A particularly important insight from the NeurIPS 2025 presentation of this research is that “many MAS failures arise from challenges in organizational design and agent coordination rather than the capability of individual agents” [10]. This shifts the diagnostic focus from the familiar terrain of LLM capability evaluation — which agent model is most accurate, which is fastest — to the less-familiar terrain of distributed systems engineering. The failure mode is often not that any single agent is insufficiently capable; it is that the coordination protocol between agents is underspecified or insufficiently robust to handle the full range of scenarios the system encounters in practice.

A counterintuitive finding further complicates the picture: successful runs are not failure-free. Analysis shows that failures occur in both successful and failed traces, suggesting that MAS succeed not by avoiding failures but by recovering from them [13]. This has profound implications for how MAS should be engineered. A system designed on the assumption that agents will always behave correctly is fragile; a system designed to detect, contain, and recover from agent failures is robust. This reframes the engineering challenge: the goal is not to build agents that never fail but to build systems with sufficient redundancy, monitoring, and error-handling to recover from the failures that will inevitably occur.

Context management has emerged as a critical and underappreciated failure vector. Cognition AI has argued that “running multiple agents in collaboration only results in fragile systems” if context engineering is not treated as a first-class concern — the reliability of MAS is fundamentally a context management problem [3]. Each agent in a MAS operates within a context window that must contain everything it needs to act correctly: its role definition, its task description, relevant history, tool specifications, and the outputs of upstream agents. As pipelines grow longer and tasks grow more complex, managing what information each agent receives — and ensuring that information is current, relevant, and correctly formatted — becomes a sophisticated engineering discipline in its own right.

LangChain has identified state management across agents, reliability in long multi-step

workflows, and debugging distributed agent interactions as the three central engineering challenges of production MAS [5]. Each of these challenges has an analog in classical distributed systems engineering: state management maps to consistency and consensus protocols; reliability in long workflows maps to saga patterns and compensating transactions; debugging distributed interactions maps to distributed tracing and observability tooling. The implication is that software engineers building MAS can draw on a rich body of distributed systems knowledge — but they must consciously apply it, rather than assuming that high-level abstractions provided by MAS frameworks handle these concerns automatically.

A final practical caution concerns the read/write asymmetry of agent operations. Best practice in MAS engineering advises: “Be cautious with write-heavy workflows. Multi-agent systems work best for read-heavy tasks and break down quickly with shared state mutations.” When multiple agents write to shared state — a shared codebase, a shared database, a shared configuration file — the potential for conflicting writes, race conditions, and data corruption grows rapidly [7]. Read-heavy workflows, where agents primarily consume and analyze information rather than modifying shared resources, are more naturally suited to concurrent multi-agent execution. This does not mean MAS cannot handle write-heavy tasks — but doing so safely requires explicit concurrency control mechanisms analogous to those used in traditional concurrent software systems.

6 Engineering Best Practices for Reliable Multi-Agent Systems

The failure modes documented in the previous chapter imply a corresponding set of engineering best practices — design principles and implementation strategies that reduce the probability and impact of failures. This chapter synthesizes guidance from production practitioners, framework developers, and research findings into a coherent set of recommendations for engineers building MAS in software engineering contexts.

The first and most important best practice is to **earn the complexity before adding agents**. The principle is clear: move to a multi-agent system only when specialization, parallel work, or orchestration clearly improves quality, speed, or maintainability — not by default [2]. Multi-agent systems are more complex than single-agent systems in every engineering dimension: they are harder to test, harder to debug, harder to monitor, and harder to reason about. A single well-prompted LLM that reliably solves 80% of your use cases is often more valuable than a sophisticated MAS that solves 85% of cases but fails unpredictably on the remaining 15%. The threshold question — does the multi-agent architecture provide a clear, measurable improvement? — should be answered affirmatively before architectural complexity is added.

Specialization with minimal overlap is the second core design principle. Each agent should have a clearly scoped role and toolset, with minimal overlap between agent responsibilities. Ambiguity at the boundaries between agents is a primary source of organizational failure modes: if two agents both believe they are responsible for a task, it may be done twice (wasting compute) or not at all (producing a gap) [5]. Role definitions should be precise enough that an agent can determine, without ambiguity, whether a given task falls within its scope. This is conceptually similar to the principle of high cohesion and low coupling in traditional software design — each module (agent) should have a focused, well-defined responsibility.

Minimal shared state is the third design principle, and it maps directly onto the warning about write-heavy workflows. Where agents must share information, that sharing should occur through explicit, immutable message passing rather than concurrent writes to a shared data structure. Shared state — a mutable artifact that multiple agents can modify — is the MAS equivalent of shared mutable state in concurrent software: a powerful but dangerous pattern that requires careful management. Architectural patterns like the blackboard pattern, in which agents write results to a shared space and read from it in a controlled, sequenced manner, offer a structured approach to shared state management that avoids the race conditions of truly concurrent writes [2].

Observability as a first-class concern is essential for production MAS. Logging all agent decisions, tool calls, and handoffs for debugging is not optional — it is the minimum viable instrumentation for understanding system behavior in production [5]. In a traditional software system, a bug can typically be traced through a call stack or a log file. In a MAS, the analogous trace spans multiple agents, potentially running concurrently, each generating their own log streams. Effective observability in MAS requires distributed tracing that correlates agent interactions across the entire pipeline, structured log formats that capture not just what an agent did but why (i.e., what reasoning it applied), and alerting systems that detect anomalous agent behavior — such as unusually long inference times, structured output format violations, or repeated tool call failures.

Fallback mechanisms and graceful degradation are the reliability engineering primitives of MAS. Agents should degrade gracefully when peers fail [5]. This means designing agents to handle cases where upstream agents produce no output, malformed output, or outputs that signal failure. A *Coder Agent* that receives a malformed specification from a *Planner Agent* should not propagate the malformation downstream; it should detect the issue, log it, and either request a retry from the *Planner* or escalate to a human operator. Retry logic with exponential backoff, circuit breakers that prevent repeated calls to failing agents, and fallback prompts that request simplified outputs when complex outputs are unavailable are all patterns worth incorporating into production MAS designs.

Google’s production guidance on LLM-based agents emphasizes the “rigorous systems engineering required to move agents from interesting prototypes to scalable, reliable production systems” — including context-window budgeting, retry logic, and structured output validation [2]. Context-window budgeting — the practice of explicitly tracking and managing the token budget available to each agent — is particularly important in long pipelines where each agent appends its output to the context that subsequent agents receive. Without budgeting, context windows fill up, forcing truncation of earlier context and producing agents that effectively forget the beginning of the task. Structured output validation — enforcing that each agent produces output in a format that downstream agents can reliably parse — prevents format mismatches from cascading into pipeline failures.

Testing multi-agent systems presents unique challenges because MAS behavior is emergent — it arises from agent interactions, not from any single component in isolation. Unit testing individual agents is necessary but not sufficient; integration tests that exercise the full agent pipeline on representative task suites are required to catch coordination failures that only manifest when agents interact. The finding that failures occur in both successful and failed traces [13] suggests that integration tests should be run repeatedly with randomized seeds, since MAS outcomes can be non-deterministic. Measuring performance across a distribution of runs, rather than relying on a single evaluation, provides

a more realistic picture of system reliability.

7 Domain Applications — From Financial Services to Production Incident Response

The practical impact of multi-agent systems in software engineering is most concretely visible in specific domains where the architectural properties of MAS — specialization, parallelism, auditability, and autonomous coordination — align well with the demands of the domain. Two domains illustrate the spectrum particularly well: financial services software engineering, where compliance and accountability requirements drive the architecture, and production operations engineering, where real-time context and autonomous response are the primary value propositions.

In financial services, software systems must satisfy a uniquely demanding combination of requirements: high correctness (errors have direct financial consequences), strict regulatory compliance (code changes may require regulatory sign-off), comprehensive auditability (every decision must be traceable), and high operational velocity (markets move faster than manual review cycles can accommodate). Multi-agent software engineering addresses this combination in a structurally natural way. Specialized AI workers can manage complex, compliance-sensitive workflows with accountability and auditability built into the agent architecture itself [9]. A financial MAS might include a Business Logic Agent that implements trading rules, a Compliance Review Agent that checks code against regulatory policy documents, a Security Audit Agent that scans for vulnerabilities, and an Orchestrator that coordinates the review pipeline and gates deployment on sign-off from each specialist agent.

The auditability property deserves particular emphasis in the financial context. In a purely human-managed workflow, demonstrating that a code change was reviewed for compliance before deployment requires documentation discipline that is difficult to enforce consistently. In a MAS architecture, the review is not merely documented after the fact — it is structurally embedded in the deployment pipeline. A Compliance Review Agent that must approve code before it reaches the Tester Agent cannot be bypassed without architectural changes that are themselves auditable. This structural enforcement of process is qualitatively different from process documentation, and it is a significant value proposition for regulated industries.

Production operations engineering presents a complementary use case. Here, the challenge is not the slow, deliberate pace of compliance review but the fast, high-stakes pace of incident response. When a production system experiences an outage or degradation, time-to-resolution directly affects business outcomes: for an e-commerce platform, every minute of downtime has a measurable revenue cost. Resolve AI has built production multi-agent systems for engineers that “maintain investigation context” on live production systems, enabling autonomous incident response [11]. In this architecture, monitoring agents continuously observe system telemetry, anomaly detection agents identify deviations from baseline behavior, investigation agents correlate signals across services to identify root causes, and remediation agents propose or execute corrective actions.

The “maintain investigation context” capability is architecturally significant. Traditional monitoring tools alert on individual signals — a CPU spike, a latency increase, an error rate anomaly — but cannot automatically synthesize these signals into a coherent narrative of what happened and why. An investigation agent with access to historical in-

cident data, current system metrics, recent deployment logs, and inter-service dependency maps can perform this synthesis, producing a root cause analysis that would otherwise require an experienced on-call engineer to construct manually. This is not about replacing the engineer's judgment; it is about presenting the engineer with a structured investigation that dramatically accelerates their ability to make a sound decision under time pressure.

The event-driven architecture identified as critical for production MAS [14] is particularly important in the operations domain. Unlike batch-oriented development workflows, production operations are inherently event-driven: alerts fire when conditions change, not on a fixed schedule. A MAS designed for production operations must respond to events within seconds, maintain consistent state across concurrent events, and handle event streams that may be sparse during normal operation or overwhelming during a major incident — without either missing critical signals or being paralyzed by alert storms. This requires careful engineering of event routing, agent load balancing, and priority queues that ensure high-severity incidents receive immediate attention even when the system is handling multiple concurrent incidents.

Beyond these two domains, MAS is finding application across the broader software engineering landscape. In developer tooling, MAS-powered assistants can provide contextually rich code suggestions that are not just syntactically correct but architecturally consistent with the project's conventions — because a dedicated Architecture Agent maintains a model of the project's structure that informs the suggestions of a Coding Assistant Agent. In software documentation, a Documentation Agent can read code, test cases, and commit messages to produce accurate, up-to-date documentation — a perennial pain point in software projects. In code migration, a MAS can orchestrate parallel translation of legacy code across multiple modules, with a Validation Agent checking that the translated code preserves the semantics of the original.

8 Trends, Evolutionary Stages, and the Changing Role of the Software Engineer

The present state of MAS in software engineering, while impressive, is widely understood to represent an early stage in a longer evolutionary trajectory. Forecasts about where MAS will be in two to five years vary in specificity, but the directional consensus is clear: the systems will become more capable, more autonomous, and more deeply integrated into the engineering workflow. Understanding this trajectory is important for software engineers who must make architectural and career decisions whose consequences will unfold over years.

A research paper on the future of AI in software engineering forecasts MAS evolution in three stages [4]. **Stage I**, representing the present state, features single powerful agents augmented with tools — essentially, capable LLMs with code interpreters, web search, and file system access. This stage is already well-established in production; tools like GitHub Copilot, Cursor, and Claude Code operate at this level. **Stage II**, projected around 2025 and already beginning to materialize, features orchestrated pipelines with specialized sub-agents — the architecture described throughout this article, where Planner, Coder, Reviewer, and Tester Agents work in coordinated pipelines. **Stage III**, projected for the 2026–2029 window, describes "Multi-Agent Teams" that mirror human engineering organizations, with agent analogs of product managers, architects, coders, testers, and

deployment engineers working together on end-to-end software projects.

The Stage III vision is ambitious and not without challenges. The coordination complexity of a multi-agent system scales roughly with the square of the number of agents (because the number of potential pairwise interactions grows quadratically), and systems with many agents require sophisticated orchestration to avoid the organizational failure modes documented in the 2025 study [13]. However, the analogy to human engineering organizations is instructive: large human engineering teams also face coordination challenges, and they address them through organizational structures (teams, chapters, guilds), communication protocols (standups, design reviews, code reviews), and shared artifacts (architecture decision records, coding standards, runbooks). The engineering challenge for Stage III MAS is to develop analogous organizational structures, protocols, and artifacts appropriate for agent teams.

The shift in the developer’s role is perhaps the most practically significant aspect of this evolutionary trajectory. A 2026 analysis notes that agentic workflows have shifted the AI coding paradigm from “pair programming” to “autonomous AI teams,” with developers acting as orchestrators rather than authors [4]. This is not a marginal shift in how developers spend their time; it is a fundamental redefinition of the software engineering role. The developer who manages a team of coding agents exercises skills of task decomposition, quality evaluation, coordination design, and exception handling — skills that overlap significantly with those of a technical lead or engineering manager. The craft of writing individual functions may diminish in importance relative to the craft of designing agent systems that reliably produce correct functions at scale.

The competitive implications of this shift are already being discussed in the engineering community. Those who can effectively manage multiple agents at scale, or design and maintain the underlying MAS infrastructure, will hold the competitive advantage [4]. Engineering organizations that invest early in the skills and tooling required to operate MAS effectively — observability infrastructure, agent evaluation frameworks, context engineering practices, and reliability patterns — will be better positioned than those that treat MAS as a productivity tool to be plugged in without architectural rethinking. This mirrors historical transitions in software engineering: the teams that invested in containerization infrastructure and CI/CD practices early gained compounding advantages over those that treated these as optional enhancements.

The maturation of formal MAS architecture standards is another important trend. The SALLMA architecture’s focus on modularity, scalability, and inter-agent communication as first-class design concerns signals that the field is moving toward the kind of principled architectural thinking that underpins reliable large-scale software systems. Just as microservices architecture provided a set of design principles for decomposing monolithic applications, formal MAS architectures will provide principled guidance for decomposing monolithic AI workflows into reliable, maintainable multi-agent systems. The emergence of these frameworks — both in open-source tooling and in formal research — suggests that MAS engineering is approaching the stage of disciplinary maturity where agreed-upon design patterns, anti-patterns, and evaluation criteria are becoming established.

Looking further ahead, the integration of MAS into the software development process may eventually blur the boundary between the development process and the software itself. A MAS designed to maintain and evolve a software system could, in principle, be part of that software system’s operational infrastructure — continuously monitoring its own outputs, detecting degradation, and initiating improvement cycles. This vision of self-maintaining software, while speculative, is a natural extrapolation of the current trajectory

and has profound implications for how software lifecycle management is conceptualized. It suggests a future in which the SDLC is not a linear sequence of human-managed phases but a continuous, autonomous improvement cycle managed by an agent team with human oversight at key decision points.

9 Synthesis — Building MAS-Ready Engineering Organizations

The preceding chapters have covered substantial technical ground — from the conceptual foundations of MAS through architectural patterns, use cases, failure modes, best practices, domain applications, and evolutionary trends. This final chapter synthesizes those threads into guidance for engineering organizations seeking to move from MAS awareness to MAS capability. The goal is not to provide a step-by-step implementation guide — the technology is moving too fast for that to remain relevant — but to identify the organizational, technical, and cultural dimensions of MAS readiness.

The first organizational requirement is a clear-eyed assessment of problem fit. Not every software engineering problem benefits from a multi-agent architecture, and organizations that deploy MAS indiscriminately will accumulate technical debt in the form of complex, fragile systems that don't deliver their promised benefits. The principle — move to MAS only when specialization, parallel work, or orchestration clearly improves quality, speed, or maintainability [2] — applies equally to engineering organizations making architectural choices as it does to individual engineers making implementation decisions. Before investing in MAS infrastructure, organizations should identify the specific bottlenecks in their software development process that a MAS architecture could address, and be prepared to measure whether the architecture actually resolves those bottlenecks.

The second requirement is investment in the infrastructure competencies that MAS demands. These include observability tooling for distributed agent systems, evaluation frameworks for assessing agent performance across diverse task distributions, context engineering practices for managing agent context windows in long pipelines, and reliability engineering patterns for building fault-tolerant agent workflows. These are not AI-specific skills imported from nowhere; they are extensions of distributed systems engineering, data engineering, and reliability engineering competencies that many engineering organizations already possess. The most effective path to MAS capability often runs through existing distributed systems expertise, augmented with specific knowledge of LLM behavior and the failure modes documented in the MAS research literature.

The choice of framework is a consequential early decision that shapes the architectural space available to the organization. The three dominant frameworks — LangGraph, AutoGen, and CrewAI — each have different production readiness profiles, community ecosystems, and integration landscapes [7]. Organizations should evaluate frameworks against their specific use cases, their existing infrastructure (Azure integration favors AutoGen; Python-native stacks align well with LangGraph; teams prioritizing rapid prototyping may start with CrewAI), and their team's existing competencies. The ALMAS [1] and SALLMA frameworks are worth monitoring as sources of architectural principles, even for organizations that do not adopt them wholesale.

Human oversight and the evolving role of the engineer merit explicit organizational attention. As developers transition from authors to orchestrators, their evaluation criteria for good work shift. A developer managing a MAS is evaluated not on the quality of the

code they personally wrote but on the quality of the agent pipeline they designed, the agent outputs they reviewed and accepted, the failure modes they anticipated and handled, and the reliability of the system they built. Organizations must update their expectations, performance metrics, and skill development programs to reflect this transition. The skills of task decomposition, quality evaluation, and agent system design should be cultivated deliberately, not assumed to emerge spontaneously.

The documentation of 14 failure modes [13] [10] provides a practical checklist for MAS quality assurance that organizations should incorporate into their evaluation processes. Specification issues, organizational and coordination issues, and agent execution issues each require different mitigation strategies: specification issues require investment in role design and task decomposition discipline; coordination issues require investment in handoff protocol design and shared state management; execution issues require investment in agent monitoring, output validation, and retry logic. A MAS review process that explicitly evaluates candidate designs against this taxonomy of failure modes will catch design weaknesses before they manifest as production incidents.

The ethical and governance dimensions of MAS in software engineering deserve acknowledgment, even if a full treatment is beyond the scope of this article. Autonomous agents making decisions about production systems — deploying code, modifying databases, responding to incidents — exercise a degree of agency that raises questions about accountability, oversight, and auditability that human-managed workflows do not. The structural auditability that MAS provides in regulated industries like financial services [9] is a model that other domains should study: building accountability into the architecture, rather than relying on documentation discipline, is a more robust approach to governance. As MAS capabilities expand and agent autonomy increases, the engineering community will need to develop clearer norms around when human approval is required before an agent action is executed, and how agent decision logs are retained and audited.

Finally, the most important insight from surveying this field is that multi-agent systems in software engineering represent a genuine paradigm shift — not the incremental improvement that many technology announcements promise, but a qualitative change in how software is conceived, built, maintained, and operated. The research evidence on failure modes [13] ensures that this assessment is grounded rather than utopian: MAS are harder to build reliably than they appear in demonstrations, and realizing their potential requires serious engineering investment. But the research evidence on capabilities [8], the framework maturity documented in production guides [7], and the evolutionary trajectory projected by leading researchers [4] collectively point to a technology that is transforming the field in ways that will compound over the coming years.

Software engineers, architects, and technical leaders who engage deeply with MAS — understanding its foundations, mastering its frameworks, studying its failure modes, and building the organizational competencies it requires — will be positioned to lead this transformation. Those who treat it as a passing trend or a plug-and-play productivity tool risk finding themselves on the wrong side of one of the most significant inflection points in the history of their profession. The stakes are high, the technology is real, and the engineering challenges are tractable for teams willing to approach them with the rigor that reliable software systems have always demanded.

10 Hebrew Summary / סיכום בעברית

מערכות רב-סוכניות בהנדסת תוכנה מודרנית

פרק זה מציג סיכום דו-לשוני של הנושאים המרכזיים במאמר.

מייצגות שינוי פרדיגמה אמיתי בהנדסת תוכנה (MAS). Instead of relying on a single monolithic model, a MAS distributes responsibilities among specialized agents — each with a defined role, memory, toolset, and communication protocol [8].

היתרונות המרכזיים כוללים: התמחות, עבודה מקבילית, ויכולת ביקורת. The key benefits are specialization, parallelism, and built-in auditability — properties that are especially valuable in regulated domains such as financial services [9].

דפוסי כישלון ייחודיים, המחולקים לשלוש קטגוריות: בעיות 14 זיהה 2025 עם זאת, מחקר משנת 2025 מצא כי רבות מהכישלונות נובעים מבעיות תיאום ארגוני, ובעיות ביצוע סוכן. However, a 2025 study found that many of these failures stem from coordination and organizational design weaknesses rather than from the capability limitations of individual agents [13].

המסקנה המעשית היא ברורה: יש לאמץ ארכיטקטורת רב-סוכנים רק כאשר התמחות, עבודה מקבילית, או תיאום מורכב מספקים יתרון ברור ומדיד. The practical conclusion is clear: adopt a multi-agent architecture only where specialization, parallelism, or complex orchestration provides a measurable and demonstrable benefit — not by default.

11 Data Visualizations

Further charts generated for this article.

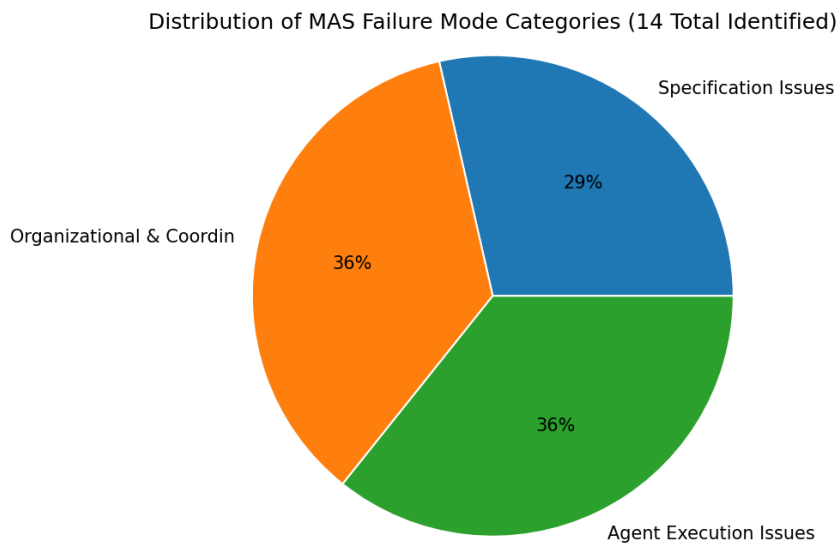


Figure 2: Distribution of MAS Failure Mode Categories (14 Total Identified)

12 Block Diagram

The diagram below shows a process described in this article.

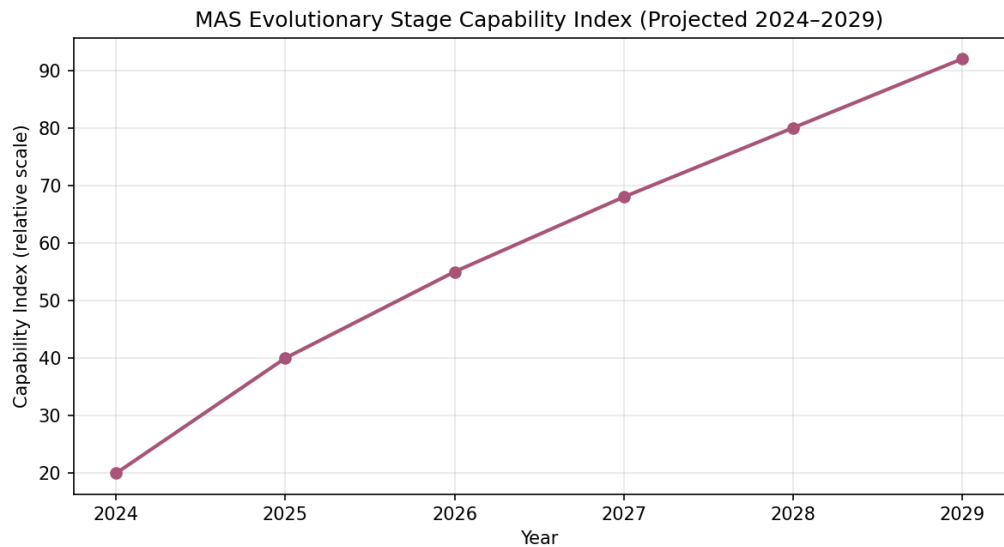


Figure 3: MAS Evolutionary Stage Capability Index (Projected 2024–2029)

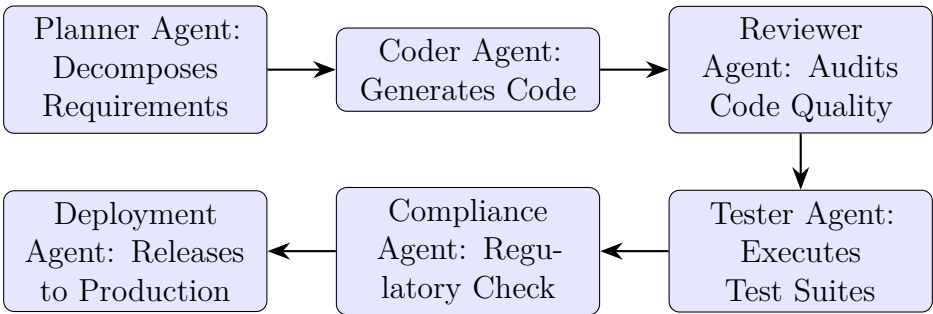


Figure 4: Typical MAS Software Engineering Pipeline: from requirements intake through autonomous deployment, showing the sequentia

References

- [1] Almas: Autonomous llm-based multi-agent software engineering — arxiv 2510.03463. <https://arxiv.org/html/2510.03463v1>, 2026. Accessed 2026.
- [2] Best practices for building effective ai agents and multi-agent systems — medium. <https://medium.com/online-inference/best-practices-for-building-effective-ai-agents-and-multi-agent-systems-2c7fe11c9605>, 2026. Accessed 2026.
- [3] Don't build multi-agents — cognition ai blog. <https://cognition.ai/blog/dont-build-multi-agents>, 2026. Accessed 2026.
- [4] The end of software engineering: How ai agents are reshaping the field — arxiv 2606.05608. <https://arxiv.org/html/2606.05608v1>, 2026. Accessed 2026.
- [5] How and when to build multi-agent systems — langchain blog. <https://www.langchain.com/blog/how-and-when-to-build-multi-agent-systems>, 2026. Accessed 2026.
- [6] How multi-agent systems are shaping the future of coding — workforceinstitute. <https://workforceinstitute.io/ai-software-engineer/how-multi-agent-systems-are-shaping-the-future-of-coding>, 2026. Accessed 2026.
- [7] Langgraph vs crewai vs autogen: Production guide (2026) — towards ai. <https://pub.towardsai.net/langgraph-vs-crewai-vs-autogen-which-ai-agent-framework-should-your-enterprise-use-in-2026-3a9ebb407b09>, 2026. Accessed 2026.
- [8] Llm-based multi-agent systems for software engineering. <https://arxiv.org/html/2404.04834v4>, 2026. Accessed 2026.
- [9] Multi-agent software engineering: Orchestrating the future of ai in financial services — medium. <https://dr-arsanjani.medium.com/multi-agent-software-engineering-orchestrating-the-future-of-ai-in-financial-services-part-2-d14cee8a4d54>, 2026. Accessed 2026.
- [10] Neurips 2025 poster: Why do multi-agent llm systems fail? <https://neurips.cc/virtual/2025/poster/121528>, 2026. Accessed 2026.
- [11] The role of multi-agent systems in making software engineers ai-native — resolve ai. <https://resolve.ai/blog/role-of-multi-agent-systems-AI-native-engineering>, 2026. Accessed 2026.
- [12] What is a multi-agent system in ai? — google cloud. <https://cloud.google.com/discover/what-is-a-multi-agent-system>, 2026. Accessed 2026.
- [13] Why do multi-agent llm systems fail? — arxiv 2503.13657. <https://arxiv.org/abs/2503.13657>, 2026. Accessed 2026.
- [14] Why multi-agent systems need real-time context in 2026 — solace (citing gartner). <https://solace.com/blog/analysts-say-mas-needs-real-time-context-edas/>, 2026. Accessed 2026.